Predict volcano eruption

Volcanic earthquakes have various characteristics depending on the volcano, and there are various classifications, but the typical classifications can be classified as follows.

Volcanic earthquake

- type A (10Hz or higher): Earthquake caused by destruction of rocks surrounding magma chambers and conduits
- type BH (5-8Hz) : Earthquake caused by magma intruding into the conduit and destroying the conduit and rocks around the conduit
- type BL (1.5-2.5Hz) : An earthquake around the conduit due to gas etc. ejecting from the crater prior to the explosive eruption and reducing the pressure inside the conduit.

Volcanic tremor

- type C (0.5-1.2Hz) : Vibration due to increase in gas pressure in the cavity along with BH
- type D (2-4Hz) : Vibration due to gas ejection along with BL

In [ ]:
```
import os
import datetime
from tqdm.notebook import tqdm

import numpy as np
import pandas as pd
import scipy
import scipy.signal

import matplotlib
import matplotlib.pyplot as plt
pd.options.display.max_columns = None    # disp all columns
%matplotlib inline

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error as mse

# from lightgbm import LGBMRegressor
import lightgbm as lgb
```

In [ ]:
```
### Kaggle or Local-PC ###
KAGGLE = True       # <==== SET ============

if KAGGLE:
    DIR = '../input/predict-volcanic-eruptions-ingv-oe'
else:               # local PC
    DIR = './predict-volcanic-eruptions-ingv-oe/'
```

Data

In [ ]:
```
train = pd.read_csv(os.path.join(DIR, 'train.csv'))
test = pd.read_csv(os.path.join(DIR, 'sample_submission.csv'))

train
```

Out[ ]:

	segment_id	time_to_eruption
0	1136037770	12262005
1	1969647810	32739612
2	1895879680	14965999
3	2068207140	26469720
4	192955606	31072429
...	...	...
4426	873340274	15695097
4427	1297437712	35659379
4428	694853998	31206935
4429	1886987043	9598270
4430	1100632800	20128938

4431 rows × 2 columns

In [ ]:
```
# Convert 'time_to_eruption'to hours:minutes:seconds (Just for reference)
train['h:m:s'] = (train['time_to_eruption']
                  .apply(lambda x:datetime.timedelta(seconds = x/100)))
train
```

Out[ ]:

	segment_id	time_to_eruption	h:m:s
0	1136037770	12262005	1 days 10:03:40.050000
1	1969647810	32739612	3 days 18:56:36.120000
2	1895879680	14965999	1 days 17:34:19.990000
3	2068207140	26469720	3 days 01:31:37.200000
4	192955606	31072429	3 days 14:18:44.290000
...	...	...	...
4426	873340274	15695097	1 days 19:35:50.970000
4427	1297437712	35659379	4 days 03:03:13.790000
4428	694853998	31206935	3 days 14:41:09.350000
4429	1886987043	9598270	1 days 02:39:42.700000
4430	1100632800	20128938	2 days 07:54:49.380000

4431 rows × 3 columns

Observe sample data

In [ ]:
```
# plot utility function
def plot(ax, x, y, xlabel=None, ylabel=None, legend=None):
    ax.plot(x, y, label = legend)
    if xlabel != None:
        ax.set_xlabel(xlabel)
    if ylabel != None:
        ax.set_ylabel(ylabel)
    if legend != None:
        ax.legend()
    ax.grid(True)
```

In [ ]:
```
# Select sample segment
sample_df = (train.sort_values('time_to_eruption')
             .reset_index()
             .rename(columns={'index': 'train_id'}))
sample_df = sample_df[sample_df.index % (len(train) // 5) == 5].reset_index(drop =
sample_ids = sample_df['segment_id'].values
sample_df
```

Out[ ]:

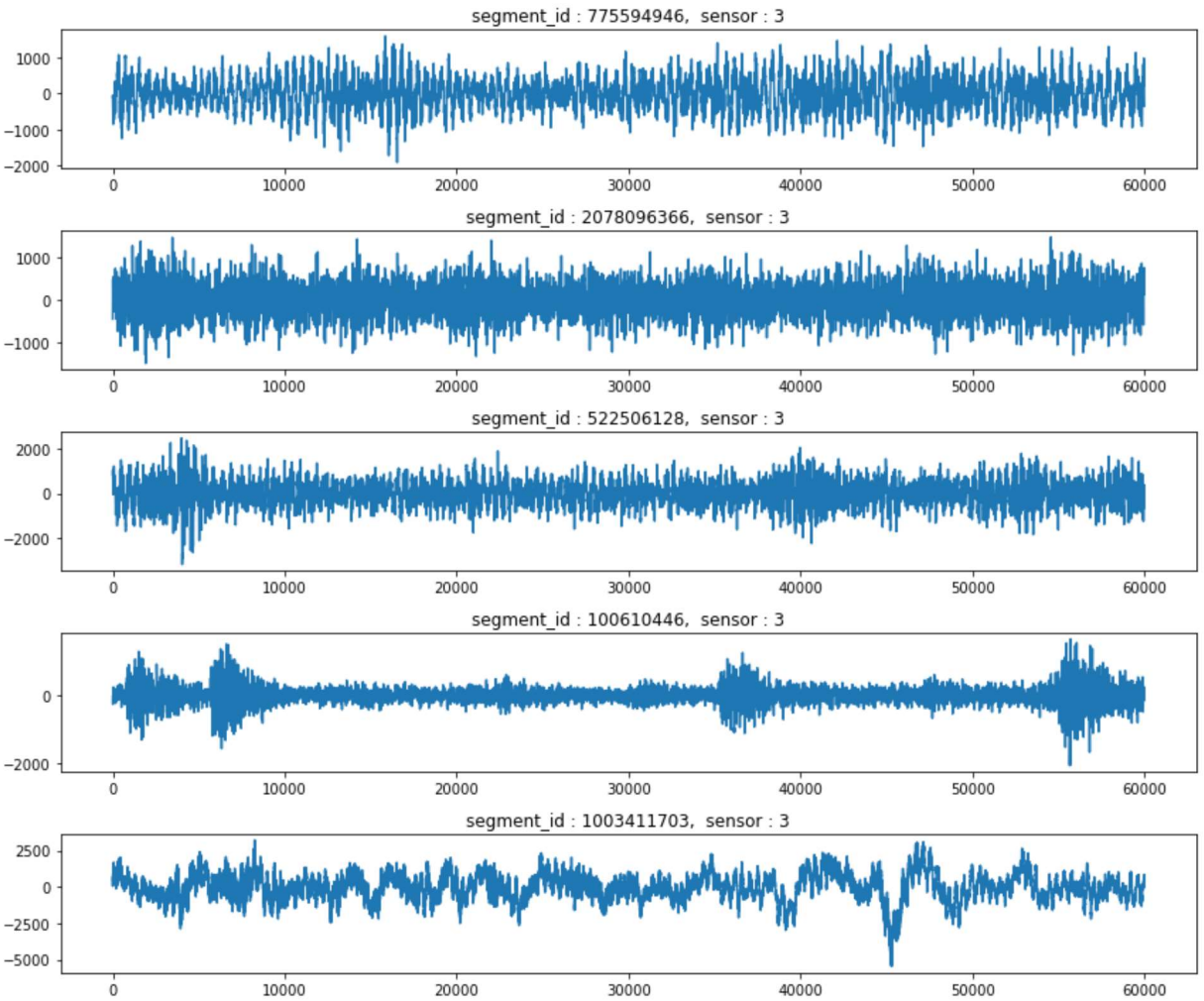
	train_id	segment_id	time_to_eruption	h:m:s
0	3942	775594946	52074	0 days 00:08:40.740000
1	636	2078096366	8991164	1 days 00:58:31.640000
2	4059	522506128	17972903	2 days 01:55:29.030000
3	3851	100610446	27292311	3 days 03:48:43.110000
4	2787	1003411703	36831703	4 days 06:18:37.030000

Time Domain

In [ ]:
```
sensor = 3     #### 1 ~ 10

fig, ax = plt.subplots(len(sample_ids), 1, figsize = (12, len(sample_ids)*2))
for i, segment_id in enumerate(sample_ids):
    segment_df = pd.read_csv(os.path.join(DIR, f'train/{segment_id}.csv')).fillna(0
    ax[i].plot(range(len(segment_df)), segment_df[f'sensor_{sensor}'])
    ax[i].set_title(f'segment_id : {segment_id},  sensor : {sensor}')

fig.tight_layout()
```



Time-Frequency Domain (STFT)

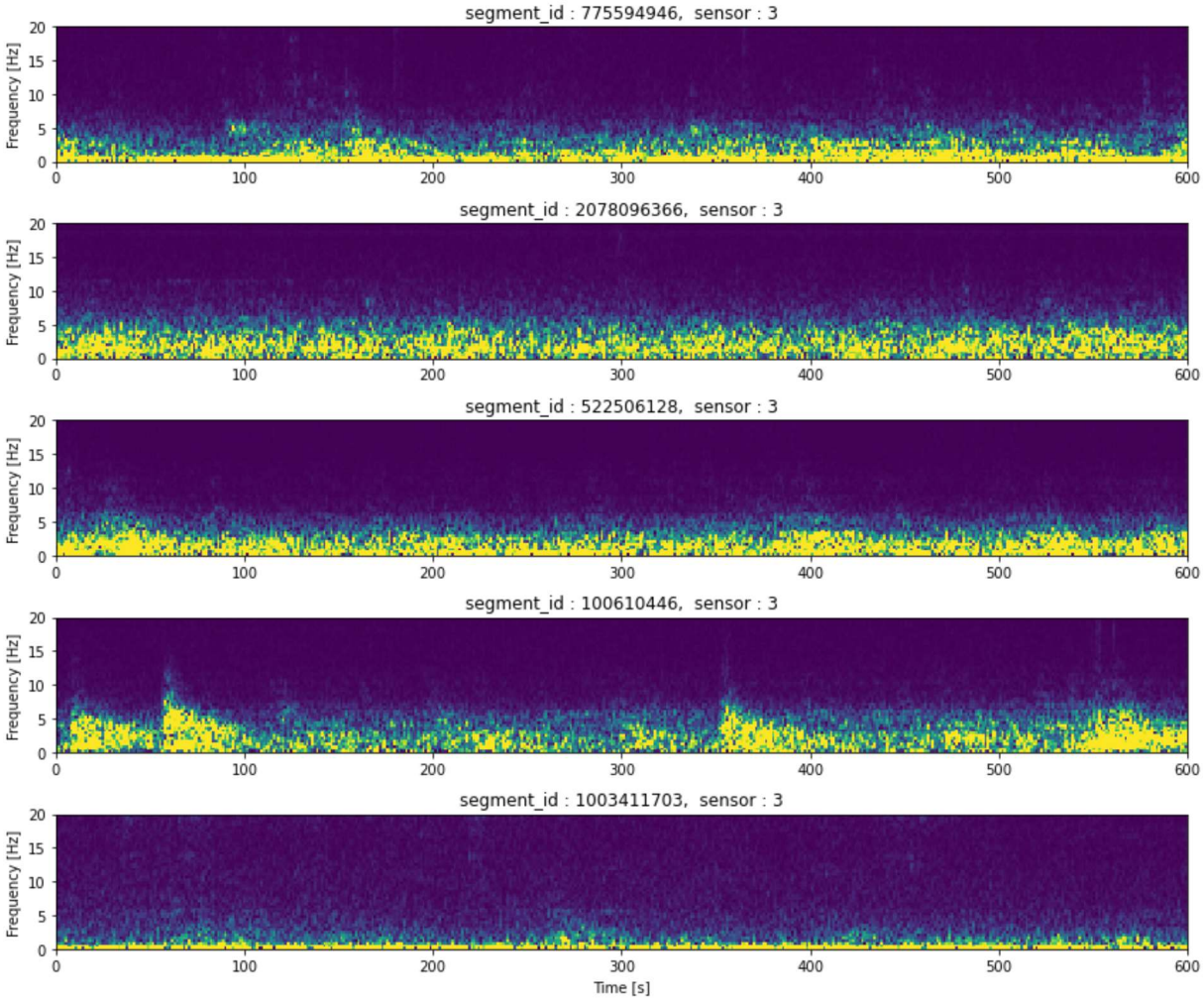
STFT : Short Time Fourier Transform

In [ ]:
```
fs = 100                # sampling frequency
N = len(segment_df)     # data size
n = 256                 # FFT segment size

fig, ax = plt.subplots(len(sample_ids), 1, figsize = (12, len(sample_ids)*2))
for i, segment_id in enumerate(sample_ids):
    segment_df = pd.read_csv(os.path.join(DIR, f'train/{segment_id}.csv')).fillna(0

    x = segment_df[f'sensor_{sensor}'][:N]
    f, t, Z = scipy.signal.stft(x, fs = fs, window = 'hann', nperseg = n)
    Z = np.abs(Z)

    ax[i].pcolormesh(t, f, Z, vmin = 0, vmax = Z.mean()*10)
    ax[i].set_ylim(0, 20)
    ax[i].set_ylabel('Frequency [Hz]'); plt.xlabel('Time [s]')
    ax[i].set_title(f'segment_id : {segment_id},  sensor : {sensor}')
fig.tight_layout()
```



Features

In [ ]:
```
# STFT(Short Time Fourier Transform) Specifications
fs = 100                # sampling frequency
N = len(segment_df)     # data size
n = 256                 # FFT segment size
max_f = 20              #  ~20Hz

delta_f = fs / n        # 0.39Hz
delta_t = n / fs / 2    # 1.28s
```

In [ ]:
```
def make_features(tgt):
    tgt_df = train if tgt == 'train' else test
    feature_set = []
    for segment_id in tqdm(tgt_df['segment_id']):
        segment_df = pd.read_csv(os.path.join(DIR,f'{tgt}/{segment_id}.csv'))
        segment = [segment_id]
        for sensor in segment_df.columns:
            x = segment_df[sensor][:N]
            if x.isna().sum() > 1000:     ##########
                segment += ([np.NaN] * 10)
                continue
            f, t, Z = scipy.signal.stft(x.fillna(0), fs = fs, window = 'hann', nper
            f = f[:round(max_f/delta_f)+1]
            Z = np.abs(Z[:round(max_f/delta_f)+1]).T    #  ~max_f, row:time,col:fre

            th = Z.mean() * 1     ##########
            Z_pow = Z.copy()
            Z_pow[Z < th] = 0
            Z_num = Z_pow.copy()
            Z_num[Z >= th] = 1

            Z_pow_sum = Z_pow.sum(axis = 0)
            Z_num_sum = Z_num.sum(axis = 0)

            A_pow = Z_pow_sum[round(10/delta_f):].sum()
            A_num = Z_num_sum[round(10/delta_f):].sum()
            BH_pow = Z_pow_sum[round(5/delta_f):round(8/delta_f)].sum()
            BH_num = Z_num_sum[round(5/delta_f):round(8/delta_f)].sum()
            BL_pow = Z_pow_sum[round(1.5/delta_f):round(2.5/delta_f)].sum()
            BL_num = Z_num_sum[round(1.5/delta_f):round(2.5/delta_f)].sum()
            C_pow = Z_pow_sum[round(0.6/delta_f):round(1.2/delta_f)].sum()
            C_num = Z_num_sum[round(0.6/delta_f):round(1.2/delta_f)].sum()
            D_pow = Z_pow_sum[round(2/delta_f):round(4/delta_f)].sum()
            D_num = Z_num_sum[round(2/delta_f):round(4/delta_f)].sum()
            segment += [A_pow, A_num, BH_pow, BH_num, BL_pow, BL_num, C_pow, C_num,

        feature_set.append(segment)

    cols = ['segment_id']
    for i in range(10):
        for j in ['A_pow', 'A_num','BH_pow', 'BH_num','BL_pow', 'BL_num','C_pow', '
            cols += [f's{i+1}_{j}']
```

```
    feature_df = pd.DataFrame(feature_set, columns = cols)
    feature_df['segment_id'] = feature_df['segment_id'].astype('int')
    return feature_df
```

In [ ]:
```
feature_df = make_features('train')
train_set = pd.merge(train, feature_df, on = 'segment_id')
train_set
```

HBox(children=(FloatProgress(value=0.0, max=4431.0), HTML(value='')))

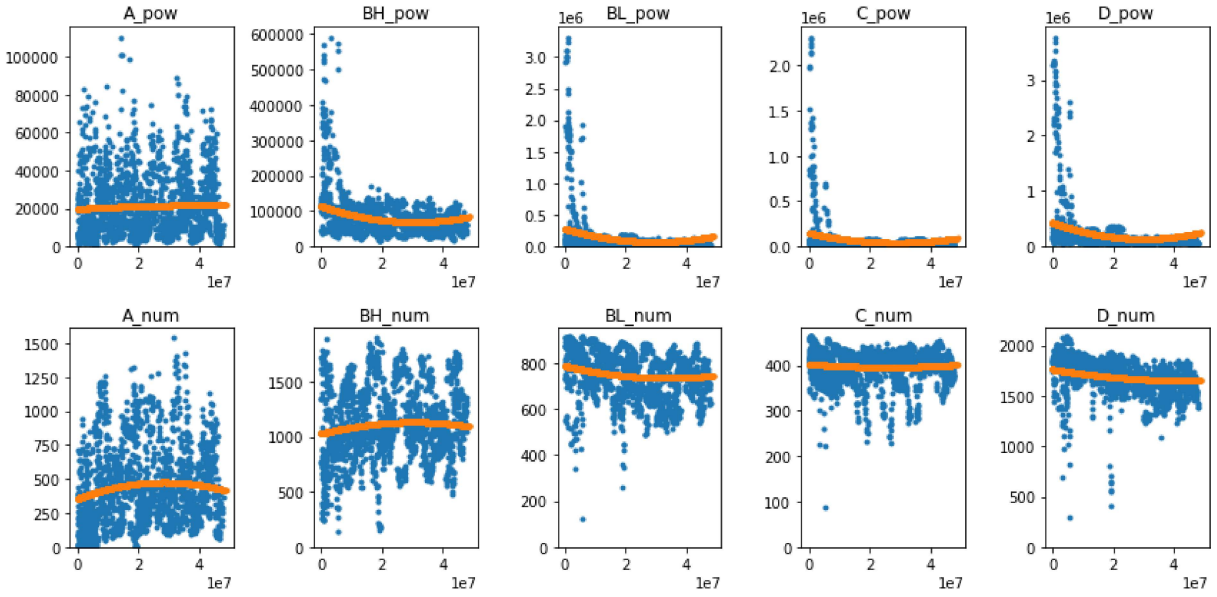
Out[ ]:

	segment_id	time_to_eruption	h:m:s	s1_A_pow	s1_A_num	s1_BH_pow
0	1136037770	12262005	1 days 10:03:40.050000	0.000000	0.0	5630.045792
1	1969647810	32739612	3 days 18:56:36.120000	0.000000	0.0	8239.546856
2	1895879680	14965999	1 days 17:34:19.990000	71.725726	5.0	8870.920230
3	2068207140	26469720	3 days 01:31:37.200000	29.114857	2.0	6797.868928
4	192955606	31072429	3 days 14:18:44.290000	0.000000	0.0	6567.271540
...	...	...	...	...	...	...
4426	873340274	15695097	1 days 19:35:50.970000	0.000000	0.0	2978.744448
4427	1297437712	35659379	4 days 03:03:13.790000	12124.863952	122.0	10834.912902
4428	694853998	31206935	3 days 14:41:09.350000	116.085819	14.0	5974.792454
4429	1886987043	9598270	1 days 02:39:42.700000	82185.532226	1776.0	5448.726206
4430	1100632800	20128938	2 days 07:54:49.380000	0.000000	0.0	8578.659270

4431 rows × 103 columns

In [ ]:
```
fig, ax = plt.subplots(2, 5, figsize = (12, 6))
x = train_set['time_to_eruption']
for i,type in enumerate(['A_pow','A_num','BH_pow','BH_num','BL_pow','BL_num','C_pow
    y = np.zeros(len(x))
    for j in range(10):
        y += train_set[f's{j+1}_{type}']
    y /= 10
    x1 = np.polyfit(x, y.fillna(y.mean()), 2)
    y1 = np.poly1d(x1)(x)
    ax[i%2, i//2].plot(x, y,'.')
    ax[i%2, i//2].plot(x, y1,'.')
```

```
    ax[i%2, i//2].set_ylim(0,)
    ax[i%2, i//2].set_title(type)
fig.tight_layout()
```



Modeling and Predicting

In [ ]:
```
df = train_set.drop(['segment_id', 'time_to_eruption','h:m:s'], axis=1)
y = train_set['time_to_eruption']

X_train, X_val, y_train, y_val = train_test_split(df, y,
                                                  random_state = 42,
                                                  test_size = 0.2,
                                                  shuffle = True)

features = X_train.columns.tolist()
cat_features = {}
```

In [ ]:
```
# lgb = LGBMRegressor(random_state = 42,
#                     max_depth = 7,
#                     n_estimators = 250,       #########
#                     learning_rate = 0.05)
# lgb.fit(X_train, y_train)
# preds = lgb.predict(X_val)

# print('RMSE: ', np.sqrt(mse(y_val, preds)))
```

In [ ]:
```
def do_lgb(X_train, y_train, X_val, y_val):
    params = {'objective': 'rmse',
              'metric': 'rmse',
              'max_depth':15,
              'min_data_in_leaf':6,         # = min_child_samples
              'num_leaves': 2**7,
              'learning_rate': 0.025,
              'feature_fraction': 0.7,      # = colsample_bytree
              'bagging_fraction': 0.5,      # = subsample
```

```
            'bagging_freq': 5,
            'lambda_l1':75,               # = reg_alpha
            'num_iterations': 10000,      # = n_estimators
            'seed': 42,
            'verbose': 1
           }

    lgb_train = lgb.Dataset(X_train, y_train)
    lgb_eval = lgb.Dataset(X_val, y_val, reference=lgb_train)

    evals_result = {}
    model = lgb.train(
        params,
        lgb_train,
        valid_sets = (lgb_train, lgb_eval),
        feature_name = features,
        categorical_feature = cat_features,
        verbose_eval = 100,
        evals_result = evals_result,
        early_stopping_rounds = 200)

    return model
```

In [ ]:
```
lgb_model = do_lgb(X_train, y_train, X_val, y_val)
```

```
Training until validation scores don't improve for 200 rounds
[100]   training's rmse: 2.30487e+06    valid_1's rmse: 4.31033e+06
[200]   training's rmse: 1.02397e+06    valid_1's rmse: 3.63186e+06
[300]   training's rmse: 651725 valid_1's rmse: 3.52189e+06
[400]   training's rmse: 451562 valid_1's rmse: 3.4846e+06
[500]   training's rmse: 329563 valid_1's rmse: 3.46086e+06
[600]   training's rmse: 256200 valid_1's rmse: 3.45429e+06
[700]   training's rmse: 213780 valid_1's rmse: 3.44897e+06
[800]   training's rmse: 173316 valid_1's rmse: 3.44537e+06
[900]   training's rmse: 134868 valid_1's rmse: 3.44495e+06
[1000]  training's rmse: 105922 valid_1's rmse: 3.44397e+06
[1100]  training's rmse: 85867.3        valid_1's rmse: 3.4434e+06
[1200]  training's rmse: 71972.3        valid_1's rmse: 3.44275e+06
[1300]  training's rmse: 60123.4        valid_1's rmse: 3.44266e+06
[1400]  training's rmse: 51101.7        valid_1's rmse: 3.44263e+06
Early stopping, best iteration is:
[1214]  training's rmse: 69869.6        valid_1's rmse: 3.44237e+06
```

Predict test data

In [ ]:
```
feature_df = make_features('test')
test_set = pd.merge(test, feature_df, on = 'segment_id')
```

In [ ]:
```
# Predict test data
preds = lgb_model.predict(test_set.drop(['segment_id', 'time_to_eruption'], axis=1)
test['time_to_eruption'] = preds
result=test[['segment_id','time_to_eruption']]
```

```
result['h:m:s']=(result['time_to_eruption'].apply(lambda x:datetime.timedelta(secon
result
```

Out[ ]:

	segment_id	time_to_eruption	h:m:s
0	1000213997	2.481368e+07	2 days 20:55:36.803664
1	100023368	3.851388e+07	4 days 10:58:58.842759
2	1000488999	2.208706e+07	2 days 13:21:10.605564
3	1001028887	2.979078e+07	3 days 10:45:07.795178
4	1001857862	1.382036e+07	1 days 14:23:23.625961
...	...	...	...
4515	996704281	2.379916e+07	2 days 18:06:31.607814
4516	997630809	1.641468e+07	1 days 21:35:46.751087
4517	998072137	2.151922e+07	2 days 11:46:32.248182
4518	998136924	3.740177e+07	4 days 07:53:37.691877
4519	99975682	3.271784e+07	3 days 18:52:58.430617

4520 rows × 3 columns

In [ ]:
```
test[['segment_id','time_to_eruption']].to_csv('submission.csv', index=False)
```